

Embedded Operating System Lab2 Report

Bo-Han Chen (陳柏翰)
Student ID:312551074
bhchen312551074.cs12@nycu.edu.tw

Q1. Shrink the size of your kernel image.

Q2: Benchmark your kernel, revise it, and improve its performance. Rerun the benchmark to prove your performance.

Shrink kernel image size - myconfig_1.0

爲了縮小 kernel 的大小，我先透過關閉一些不必要的功能和模組來實現，本次 Lab 使用 bcm2711_defconfig 作爲預設的設定檔和後續比較大小的對象，並透過 make menuconfig 來進行設定。在第一次嘗試中，我在 make menuconfig 提供的 Device Driver 選項中，關閉了以下不須用到的模組：

- Input Device support
 - Joysticks/Gamepads
 - Keyboards
 - Mouse interface
 - Touchscreens
 - Miscellaneous devices
- Multimedia support
- Graphics support
- Sound card support

因爲 make menuconfig 提供的選項非常多，沒辦法一一確認關閉後是否會對系統造成影響，所以我參考了 Kernel Size Tuning Guide 網站，裡面提供了 Linux-tiny (一個專門用來縮小 kernel 大小的專案) 和預設設定檔的差異。另外網站中也提到了可以使用 `nm --size -r vmlinux` 來取得 kernel 中占用較多容量的符號 (symbol)，因此我先取得了 bcm2711_defconfig 容量前 10 大的符號：

```
1 $ nm --size -r vmlinux.bcm2711_defconfig | head -10
2 00000000000058000 d _printk_rb_static_infos
3 00000000000040000 b fscache_cookie_hash
4 00000000000038630 B g_state
5 00000000000020000 b __log_buf
6 00000000000018000 d _printk_rb_static_descs
7 00000000000010000 b stack_slabs
8 00000000000009000 B cpu_topology
9 00000000000008000 d ftrace_stacks
10 00000000000007620 d sys_reg_descs
11 00000000000006000 b memblock_memory_init_regions
```

可以看到與 printk 相關的符號佔用了很大的空間，因此可以根據以上資訊關閉了下列選項：

- CONFIG_PRINTK
- CONFIG_BUG
- CONFIG_ELF_CORE
- CONFIG_AIO

- CONFIG_SHMEM
- CONFIG_POSIX_MQUEUE
- CONFIG_LOG_BUF_SHIFT
- CONFIG_CC_OPTIMIZE_FOR_PERFORMANCE
- CONFIG_SCSI

並開啓

- CONFIG_CC_OPTIMIZE_FOR_SIZE

透過這些設定，kernel 大小可以從原先 2711_defconfig 的 23MB (23589376 bytes) 縮小到 myconfig_1.0 的 19M (19390976 bytes)。

Benchmark the kernel

我使用了 Unixbench 來進行系統效能測試，每次測試都包含下列項目並進行單執行緒和多 (4) 執行緒的測試:

- Dhrystone 2 using register variables
- Double-Precision Whetstone
- Exec1 Throughput
- File Copy (30 seconds)
- Pipe Throughput
- Pipe-based Context Switching
- Process Creation
- Shell Scripts (1 concurrent)
- Shell Scripts (8 concurrent)
- System Call Overhead
- System Benchmarks Index Score

Unixbench 會根據這些項目的測試結果，計算出單核和多核兩個分數，分數越高代表效能越好。接下來我測試了原先 bcm2711_defconfig 和經過縮小後的 myconfig_1.0 的效能，結果如下表格所示:

Kernel Config	Single Core	Multi Core
bcm2711_defconfig	171.9	533.9
myconfig_1.0	122.9	308.8

Table 1: Unixbench Score - bcm2711_defconfig VS myconfig_1.0

可以發現經過縮小後的 myconfig_1.0 在單核和多核的效能都有所下降，這是因為我關閉了一些模組和功能所導致，因此我會在下一小節進行改善，希望可以在縮小 kernel 的同時提升其效能。

Shrink kernel image size - myconfig_2.0

爲了避免關閉一些模組和功能導致效能下降，我重新檢視了 version_1.0 中會影響效能的設定並修正，並且關閉了更多不必要的模組和功能。

首先我修正了以下設定：

- CONFIG_CC_OPTIMIZE_FOR_PERFORMANCE
- CONFIG_CC_OPTIMIZE_FOR_SIZE

CONFIG_CC_OPTIMIZE_FOR_PERFORMANCE 因爲使用 gcc -O2 flag，會導致 kernel 使用更多的空間，但可以提升 Network Stack、File System 和 Memory Management 等方面的效能；而 CONFIG_CC_OPTIMIZE_FOR_SIZE 則是針對 kernel 的大小進行最佳化，但可能會導致效能下降。因此在折衷之下，我選擇了 CONFIG_CC_OPTIMIZE_FOR_PERFORMANCE 並尋找其他不必要的模組來縮小 kernel：

- Device drivers
 - Amateur Radio support
 - Bluetooth subsystem
 - Remote Controller support
 - Accessibility support
 - Microsoft Surface Platform-Specific Device Drivers
 - Wireless LAN
- File systems
 - CONFIG_JFS_FS
 - CONFIG_XFS_FS
 - CONFIG_BTRFS_FS
 - CONFIG_HFS_FS
 - CONFIG_HFSPLUS_FS
 - CONFIG_CIFS
 - CONFIG_NFS_FS

透過這些設定，myconfig_2.0 的 kernel 大小爲 19MB (19663360 bytes)，並在 Unixbench 效能測試中取得單核 175.2 和多核 543.7 的分數，最終整理爲下表：

Kernel Config	Kernel Size (bytes)	Unixbench Score (Single Core / Multi Core)
bcm2711_defconfig	23589376	171.9 / 533.9
myconfig_1.0	19390976	122.9 / 308.8
myconfig_2.0	19663360	175.2 / 543.7

Table 2: Kernel Size and Unixbench Score Comparison

可以看到經過改善後的 myconfig_2.0 在 kernel 大小和效能上都有所提升，因此我認爲這是一個比較好的設定。

Discussion

在 Q1 和 Q2 的實作中，我學習到了如何透過 make menuconfig 來詳細設定 kernel，刪減不必要的模組並縮小 kernel 以便在嵌入式系統中使用，減少不必要的資源浪費；另外也學習到了如何使用 Unixbench 來進行系統效能測試，並透過測試結果來改善 kernel 的效能，過程中查到許多關乎 kernel 效能的設定，但因爲多數設定都已經爲 2711_defconfig 預設，且時常遇到改動設定後無法進行編譯或無法開機的問題，因此我認爲在改善 kernel 效能上還有很多需要學習的地方。

Q3: Patch your kernel to support real-time tasks.

在 Q3 中，我使用了 kernel 4.18.20 並透過 patch-4.18.12-rt7 來支援 real-time tasks，首先將 patch 檔案放置在 kernel 的根目錄下並執行以下指令：

```
1 $ cat patch-4.18.12-rt7 | patch -p1
```

因為 kernel 版本和 Q1、Q2 不同，沒有 2711_defconfig 設定檔，因此要先進入 linux/arch/arm64/configs 尋找可作為預設的設定檔生成 .config (我使用的是 bcmrpi3_defconfig)，接著 make menuconfig 開啓 Preemption Model > Fully Preemptible Kernel (RT) 選項，並進行編譯，我在編譯過程中有遇到 multiple definition of `yylloc` 的報錯，透過查找網路上的解決方法後將 yylloc 定義的地方加上 extern，即可順利編譯完成。燒錄到 SD 卡後，我使用了 uname -a 來確認 kernel 版本和是否支援 real-time tasks:

```
1 $ uname -a
2 Linux raspberrypi 4.18.20-rt7-v8+ #1 SMP PREEMPT RT Wed Oct 2 20:46:53 CST
   2024 aarch64 GNU/Linux
```

可以看到 kernel 版本為 4.18.20-rt7 並支援 real-time tasks。

References

1. Kernel Size Tuning Guide
2. Linux-tiny
3. byte-unixbench
4. linux kernel rt patch
5. 編譯 linux 內核時 multiple definition of `yylloc` 錯誤的解決方案
6. 編譯 PREEMPT_RT 核心與安裝
7. Realtime Linux
8. What are the differences between make clean, make clobber, make distclean, make mrproper and make realclean?